

printf(format, arg1, arg2, ...)

`format` – управляющий C-string (в двойных кавычках), содержащий:

- обычные или управляющие символы: `text \n \' \' %% ...`
- спецификаторы преобразования: `%...specifier`

specifier	ожидаемый тип	примечание и пример
d,i	signed int	десятичные: -10
u	unsigned int	десятичные: 10
o	unsigned int	восмеричные: 012
x,(X)	unsigned int	шеснадиатиричные: d1ff (D1FF)
f	double или float	0.123456
e,(E)	double или float	1.234567e-01 (or 1.234567E-01)
g,(G)	≡ %e (%E) или %f, какой короче	
c	char	один символ: c
s	C-string	текст: text
p	void*	указатель: 0x220a1b0

Модификаторы: на месте точек в % ...specifier может стоять

- + (знак плюс) — печать со знаком
- - (знак минус) — выравниванию по левому краю
- w.p — ширина(w): полное количество символов в числе
точность(p): количество знаков после точки
- модификаторы длины переменной:

	sizeof	specifiers	example
h	short int	d i o u x	%hd
l(ell)	long int	d i o u x	%li
ll(ell-ell)	long long int	d i o u x	%llu
L	long double	e f g	%Lf

Управляющие символы

\ " double quote
\ a alert (BEL)
\ r carriage return
\ t horizontal tab

\\ backslash
\ b backspace
\ n new line
%% a single %

scanf(format, arg1, arg2, ...)

Обратите внимание

- в функцию `scanf()` необходимо передавать адреса переменных
- обычные символы в `format` проверяются на совпадение с входным потоком и в случае несовпадения чтение прекращается
- значения спецификаторов не полностью совпадают с `printf`:
 - ✓ `%i` – читает целое число в десятичном, восьмеричном и шестнадцатеричном формате
 - ✓ `%f %e %g` – соответствуют данным типа `float`
 - ✓ `%lf %le %lg` – для `double`
 - ✓ `%Lf %Le %Lg` – для `long double`

Возвращаемые значения

- `printf()`:

- ✓ число «напечатанных» символов
- ✓ отрицательное значение в случае ошибки потока вывода или ошибки преобразования строк или символов

- `scanf()`:

- ✓ число успешно «прочитанных» аргументов: может быть нуль, если ошибка сопоставления произошла до присвоения значения первому аргументу
- ✓ **EOF**, если конец ввода случился **перед первым аргументом**